Real-Time MIDI Call-and-Response Generation Using Autoregressive Transformers

Wang Zitong, Hu Sitong

June 2026

Abstract

This study asks whether an offline autoregressive Transformer can be made usable for real-time MIDI "Call-and-Response" co-performance without retraining. It proposes an external adaptation architecture that combines MIDI-VAD phrase segmentation, phrase-level musical control, asynchronous scheduling, and speculative preload around an Anticipatory Music Transformer. Evaluation uses a three-layer evidence chain: a 27,000-trial Call100 comparison among raw AMT, controlled AMT, and a motif-transformation baseline; a 63,000-row A0–A6 module ablation; and an 18,000-row preload latency experiment. Controlled AMT improves substantially over raw AMT under objective structural metrics, while the motif baseline remains a strong comparison. Full controlled AMT raises mean objective score from 0.560968 to 0.732610, and speculative preload reduces mean endpoint-to-first-MIDI latency from 161.303 ms to 85.534 ms while lowering buffer underrun rate from 20.744% to 3.556%. These results support structural controllability and local runtime responsiveness; listening tests and live-performance user studies remain future work.

Keywords: symbolic music generation; MIDI; call-and-response; Transformer; temporal point process; real-time interaction; objective evaluation; ablation study; latency logging; speculative preload

1 Introduction

Call-and-response is a foundational pattern in live improvisation, but realizing it computationally requires symbolic generators that can react on a performance clock rather than only complete offline continuations.

Autoregressive Transformers capture rich symbolic-music structure in offline generation, yet their stepwise decoding is poorly matched to live co-performance. Direct deployment creates four linked bottlenecks: synchronization with human timing, reliable phrase-end detection, latency spikes during inference, and structural degradation in high-entropy continuations. This paper therefore asks whether these limits can be reduced through external architecture and control layers without retraining the underlying model.

The main contribution is an adaptation method around an offline Anticipatory Music Transformer (AMT). A continuous-time stopping-time formulation defines turn taking; a closed-loop MIDI system implements routing, asynchronous decoding, micro-buffered playback, phrase-level control, and speculative preload; and Call100, A0–A6, and latency experiments evaluate the method at candidate, module, and scheduler levels.

2 Related Work

This section reviews the four areas that shape the proposed system: symbolic music representations, generative sequence models, real-time co-performance systems, and evaluation of generative music.

2.1 Symbolic Music Representation

Early symbolic music generation often used piano-rolls or fixed time-step grids, but rigid quantization can create sparse matrices and discard micro-timing such as rubato and swing [1]. Event-based representations address this limitation through MIDI-like streams with `Note-On`, `Note-Off`, and relative `Time-Shift` events [10, 5]. The Anticipatory Music Transformer further models symbolic music with temporal point-process events and asynchronous controls [12]. However, representation-level advances alone do not specify how continuous MIDI input should be segmented into interactive turns, especially when chords, hesitation pauses, and rubato-like timing occur in live performance.

2.2 Sequence Models for Music Generation

Symbolic music generation has moved from memoryless or recurrent models such as LSTMs [3] to self-attention-based Transformer architectures [13, 5]. AMT extends this line toward controllable temporal-event generation [12], but it remains primarily an offline model. Live co-performance adds endpoint detection, turn scheduling, and latency stability as deployment constraints. These models therefore do not directly address phrase-end detection or scheduler-level latency, which become necessary when offline symbolic generation is embedded into live interaction.

2.3 Real-Time Human-AI Co-Performance

Real-time human-AI co-performance systems must handle input capture, turn-taking, latency, and synchronization in addition to generation quality. MIDI-performance research emphasizes timing stability in interactive systems [9]; Scramble Live illustrates the combination of learned models and live control in Max/MSP-style environments [11]; and recent work couples Max/MSP front ends with Python generative back ends [7]. These systems show the importance of live control surfaces, but they leave open how an offline autoregressive symbolic model can be wrapped with endpoint logic, buffering, and fallback behavior without retraining. This study addresses that deployment gap through virtual MIDI routing and multithreaded asynchronous scheduling.

2.4 Evaluation of Generative Music

While human listening tests remain vital for evaluating musical quality, they are expensive and difficult to scale. Prior surveys emphasize that objective metrics for symbolic and generative music are useful for reproducible structural analysis, but they cannot fully replace perceptual or human-centered evaluation [6, 8]. For real-time call-and-response, evaluation must also separate structural response quality from runtime behavior such as endpoint-to-first-MIDI latency and buffer underrun. The present evaluation therefore treats objective metrics, ablation evidence, and scheduler logs as complementary rather than interchangeable forms of evidence.

3 Temporal Point Process Modeling and Autoregressive Generation

3.1 Continuous-Time MIDI Event Representation

Fixed-step piano-roll representations can obscure the micro-timing that matters in improvisation. This study therefore represents MIDI performance as a continuous-time event stream.

$$E = \{e_1, e_2, \ldots, e_N\}. \tag{1}$$

Each event e_i is represented as a triplet:

$$e_i = (t_i, d_i, p_i), \tag{2}$$

where

$$t_i \in \mathbb{R}^+ \tag{3}$$

denotes the arrival time of the event and satisfies the non-strictly increasing temporal constraint

$$t_1 \leq t_2 \leq \cdots \leq t_N. \tag{4}$$

The non-strict temporal ordering allows block chords and other near-simultaneous sonic events. Section 4 later applies chord clustering so that such events do not inflate onset-density estimates.

Furthermore,

$$d_i \in \mathbb{R}^+ \tag{5}$$

denotes the duration of the note, and

$$p_i \in \{0, 1, \ldots, 127\} \tag{6}$$

denotes the absolute MIDI pitch under the twelve-tone equal temperament system.

This representation turns polyphonic and cross-tonal performance into a unified event-sequence input for autoregressive Transformer modeling.

3.2 Conditional Autoregressive Factorization

In call-and-response interaction, the system generates an AI Response sequence R conditioned on the human Call sequence C:

$$C = \{e_1^c, e_2^c, \ldots, e_m^c\} \tag{7}$$

$$R = \{e_1^r, e_2^r, \ldots, e_n^r\} \tag{8}$$

The generation objective is to find a likely continuation under the Call:

$$R^* = \arg\max_R P(R \mid C). \tag{9}$$

Following the self-attention architecture of Vaswani et al. [13], a causally masked Transformer decomposes the conditional probability over response events:

$$P(R \mid C) = \prod_{j=1}^{n} P(e_j^r \mid C, e_{<j}^r). \tag{10}$$

where

$$e_{<j}^r = \{e_1^r, e_2^r, \ldots, e_{j-1}^r\} \tag{11}$$

denotes the previously generated response history. Each response event is again represented by arrival time, duration, and pitch:

$$e_j^r = (t_j^r, d_j^r, p_j^r). \tag{12}$$

The single-event distribution is factorized locally as

$$P(e_j^r \mid \cdot) = P(t_j^r \mid \cdot) P(d_j^r \mid t_j^r, \cdot) P(p_j^r \mid t_j^r, d_j^r, \cdot). \tag{13}$$

Thus the model predicts event timing, duration, and pitch in sequence. A lower-triangular causal mask prevents access to future positions, which is necessary for streaming continuation but insufficient for musical coherence or controllability. Section 4 therefore adds endpoint detection, nucleus sampling, phrase-level control, and optional style constraints to reduce repetitive loops, tonal drift, rhythmic imbalance, and empty outputs.

3.3 Dynamic Turn Segmentation Based on Continuous-Time Stopping Time

In free-form Call-and-Response interaction, accurately detecting the endpoint of a human-performed phrase is a prerequisite for seamless musical continuation. Instead of relying on rigid bar-line segmentation or fixed metrical clocks, this study models the human performer's free Note-On events as stochastic arrivals on a continuous time axis. Specifically, the MIDI Note-On event stream is formulated as a time-dependent inhomogeneous Poisson process. This formulation follows standard temporal point-process theory [2].

Assume that within the time interval $[0, t)$, the MIDI event stream generated by the performer is governed by a time-varying intensity function $\lambda(t)$. When the performer is actively playing, the inter-arrival time between adjacent events,

$$\Delta t_i = t_i - t_{i-1}, \tag{14}$$

is determined by the local performance rate. A higher local onset density corresponds to a higher event-arrival intensity, while a lower onset density indicates a slower or more sparse performance gesture.

To dynamically estimate whether a human phrase has ended, this study introduces the survival-function formulation from survival analysis. Let T_{last} denote the timestamp of the most recent Note-On event. After this event, the relative silence duration is defined as

$$\tau = t_{\text{current}} - T_{\text{last}}. \tag{15}$$

The survival function $S(\tau)$ is defined as the probability that no new event arrives within the silence interval of length τ, or equivalently, that the next inter-arrival time exceeds τ:

$$S(\tau) = P(\Delta t > \tau). \tag{16}$$

According to the properties of an inhomogeneous Poisson process, the probability that no event occurs within the interval $[T_{\text{last}}, T_{\text{last}} + \tau]$ is determined by the integral of the intensity function. Therefore, the survival function can be written as

$$S(\tau) = P(\Delta t > \tau) = \exp\left(-\int_0^{\tau} \lambda(T_{\text{last}} + u)\, du\right). \tag{17}$$

Here, the corresponding hazard function can be interpreted as the instantaneous rate at which a new Note-On event may arrive after the performer has remained silent for duration τ:

$$h(\tau) = \lambda(T_{\text{last}} + \tau). \tag{18}$$

Therefore, the hazard function is not interpreted as a direct "death risk" of the performer continuing to play. Instead, as the silence duration τ increases without observing any new Note-On event, the system's confidence that the current phrase has ended gradually increases under the local arrival model.

For practical real-time implementation, the local intensity function is approximated by the recent onset density of the performer. Let μ_{tempo} denote the estimated local performance rate. By assuming that the event-arrival intensity remains approximately constant within a short local window, we set

$$\lambda(t) \approx \lambda_0 = \mu_{\text{tempo}}. \tag{19}$$

The survival function can then be simplified as

$$S(\tau) = \exp(-\mu_{\text{tempo}}\tau). \tag{20}$$

This equation indicates that, under the local constant-rate approximation, the probability of observing no new event for a silence duration τ decays exponentially as τ increases.

To make an endpoint decision, the system introduces a confidence boundary θ, where $0 < \theta < 1$. Instead of claiming that the system can determine with absolute certainty that the human performer has stopped playing, the endpoint is detected when the survival probability falls below the predefined confidence boundary:

$$S(\tau) \leq \theta. \tag{21}$$

Equivalently, the system makes an endpoint decision under a given confidence boundary when

$$\exp(-\mu_{\text{tempo}}\tau_{\text{cutoff}}) = \theta. \tag{22}$$

Taking the natural logarithm on both sides gives

$$-\mu_{\text{tempo}}\tau_{\text{cutoff}} = \ln(\theta). \tag{23}$$

Thus, the dynamic cutoff threshold can be derived as

$$\tau_{\text{cutoff}} = -\frac{\ln(\theta)}{\mu_{\text{tempo}}}. \tag{24}$$

Using the identity $-\ln(\theta) = \ln(1/\theta)$, the final form becomes

$$\tau_{\text{cutoff}} = \frac{\ln(1/\theta)}{\mu_{\text{tempo}}}. \tag{25}$$

The resulting stopping-time rule can therefore be expressed as

$$t_{\text{current}} - T_{\text{last}} \geq \tau_{\text{cutoff}}. \tag{26}$$

When this condition is satisfied, the system treats the current Call phrase as completed and triggers the subsequent response-generation module. Since τ_{cutoff} is inversely proportional to μ_{tempo}, the threshold adapts to the performer's local playing speed: faster passages lead to a shorter allowable silence duration, while slower passages produce a longer endpoint-detection tolerance. This adaptive formulation allows the system to segment musical turns in a continuous-time manner without relying on fixed metrical boundaries.

3.4 Section Summary

This section establishes two foundations for real-time MIDI call-and-response: a continuous-time event representation for symbolic performance, and a stopping-time endpoint rule based on the silence after the latest Note-On event. The resulting cutoff,

$$\tau_{\text{cutoff}} = \frac{\ln(1/\theta)}{\mu_{\text{tempo}}}, \tag{27}$$

adapts phrase segmentation to local playing speed instead of fixed bar lines or fixed silence thresholds. Section 4 translates this model into engineering mechanisms for chord clustering, revocable endpoint decisions, asynchronous decoding, speculative preload, and phrase-level control.

4 Real-Time MIDI Call-and-Response System Design

4.1 Cross-Environment Architecture Based on Virtual MIDI

The system uses a modular Virtual MIDI Bus architecture rather than a monolithic offline-generation pipeline. Physical MIDI input, host audio rendering, neural inference, and MIDI output scheduling are decoupled so that equivalent routing chains can be used across operating systems and digital audio workstations.

In Windows, the routing chain can use loopMIDI with REAPER, Decent Sampler, or another host; in macOS, it can use the IAC Bus with Logic Pro or an equivalent DAW. The system is therefore defined by the interaction chain itself: physical MIDI input, virtual MIDI routing, a Python inference engine, and host-based rendering.

The overall structure of the system can be split into two main components.

1. **Front-end performance capture and acoustic rendering component.**

   This component captures human key events through the MIDI keyboard and virtual ports, then renders returned AI MIDI through a DAW or sampler such as REAPER, Logic Pro, or Decent Sampler.

2. **Back-end intelligent control and inference component.**

   This Python component receives the event stream, maintains the input window, detects endpoints, runs asynchronous inference, applies sampling control, and schedules playback without blocking live input.

The workflow has four phases, summarized in Figure 1: MIDI capture, dynamic endpoint detection, asynchronous Transformer generation, and micro-buffered playback. Logging and offline objective evaluation are shown as a separate analysis path.

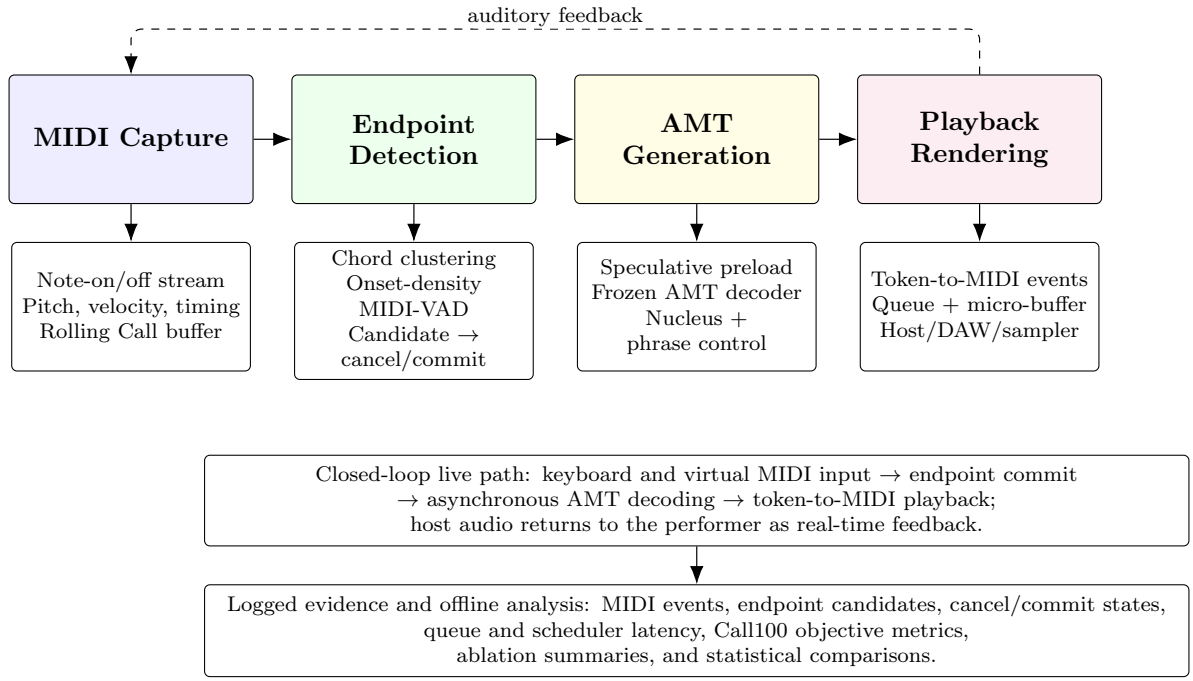

Figure 1: System overview of the real-time MIDI call-and-response framework. The figure shows the four online stages, the auditory feedback loop, and the logged evidence used for offline evaluation.

Phase 1: Listening Phase.

Front-end MIDI input streams into the Python engine, which updates a sliding Call window with Note-On timestamps, pitches, and durations without triggering generation.

Phase 2: Endpoint Detection Phase.

The MIDI VAD module estimates local onset density and the dynamic cutoff threshold. When the silence condition is satisfied, the current input window becomes a candidate completed Call.

Phase 3: Overlapped Inference Phase.

The confirmed Call prompts the pretrained Transformer. Asynchronous threads, a thread-safe queue, and a micro-buffer overlap decoding and playback scheduling, while nucleus sampling and later control layers improve response stability.

Phase 4: Seamless Playback Phase.

Generated event tokens are converted into MIDI and routed to the host audio environment, completing one closed-loop turn from human Call to AI Response playback.

This separation of inference, MIDI monitoring, endpoint detection, and rendering improves portability while providing the basis for endpoint correction, streaming decoding, preload, and phrase-level control.

4.2 Engineering Implementation of the Intelligent MIDI Endpoint Detection Module

The Python back end implements the stopping-time rule as a MIDI VAD module. Its task is to detect whether the current Call has ended using local playing speed and real-time silence duration rather than fixed bars or fixed pauses.

For clarity, the control logic of the MIDI VAD module can be abstracted as the following state machine:

$$\text{Listening} \rightarrow \text{Candidate} \rightarrow \begin{cases} \text{Cancel} \rightarrow \text{Listening}, \\ \text{Commit} \rightarrow \text{Inference}. \end{cases} \tag{28}$$

Here, *Listening* monitors MIDI input, *Candidate* marks an unconfirmed endpoint, *Cancel* revokes it after a new Note-On event, and *Commit* freezes the Call and starts inference.

In implementation, a daemon thread polls the MIDI input port. Each Note-On updates the latest event time T_{last}, while a sliding window of recent onsets estimates μ_{tempo}. Using the dynamic threshold from Section 3,

$$\tau_{\text{cutoff}} = \frac{\ln(1/\theta)}{\mu_{\text{tempo}}}, \tag{29}$$

the threshold adapts to current playing speed. When the silence duration satisfies

$$t_{\text{current}} - T_{\text{last}} > \tau_{\text{cutoff}}, \tag{30}$$

the system enters the candidate state and records $T_{\text{candidate}}$ rather than closing the input immediately. A short confirmation window then distinguishes true phrase endings from temporary pauses, turning the stopping-time model into a real-time segmentation mechanism.

4.2.1 Chord-Clustering Correction for Onset-Density Estimation

Block chords, rapid arpeggios, and near-simultaneous notes can distort the Poisson-process endpoint model. If every physical Note-On is counted as an independent onset, μ_{tempo} is inflated, τ_{cutoff} becomes too short, and the system may prematurely close a phrase after a chord or dense gesture.

To address this issue, the system introduces a chord-clustering time window in the MIDI VAD computation layer:

$$\Delta t_{\text{cluster}} \approx 0.08\,\text{s}. \tag{31}$$

Note-On events inside this window are merged into one onset cluster, correcting the density estimate from physical-note density to musical-onset density. Chord clustering therefore preserves the stopping-time model while making endpoint decisions less sensitive to chordal textures and fast attack gestures.

4.2.2 Two-Stage Revocable Candidate Endpoint and Confirmation Window

Human performers often use pauses inside unfinished phrases, especially after syncopated gestures or expressive breaths. To avoid early AI entry, the endpoint rule is implemented as a revocable two-stage decision: candidate marking followed by confirmation-window waiting.

When Equation (30) is first satisfied, the system marks the candidate time

$$T_{\text{candidate}} = t_{\text{current}}, \tag{32}$$

and enters the *Candidate* state. It then opens a short confirmation window:

$$\Delta T_{\text{confirm}} \approx 0.15\,\text{s}. \tag{33}$$

If a new Note-On arrives during this window, the candidate is canceled and the original Call continues. If no event arrives, the candidate is committed, the Call window is frozen, and asynchronous inference begins. This buffer reduces false endpoints during fast passages, syncopation, and hesitation pauses, and it also creates the window used by speculative preload.

4.3 Asynchronous Streaming Decoding and Micro-Buffered Scheduling

Conventional autoregressive generation waits for the full phrase before playback, which creates noticeable delay in real-time call-and-response. The system instead uses asynchronous streaming decoding with a micro-buffer: generated events are pushed to playback as soon as they become available, allowing inference and acoustic playback to overlap.

The implementation uses two parallel threads and a thread-safe queue.

(1) Inference Thread. After endpoint confirmation, a background inference thread runs Transformer decoding and writes each new event token $e_i = (t_i, d_i, p_i)$ to the shared queue.

(2) Playback Thread. The playback thread monitors the queue, converts playable tokens into MIDI, and sends them through the virtual MIDI bus to the host renderer.

To reduce buffer underrun, playback starts after a short initial window ΔT_{buffer}, typically 80 ms.

Let the single-step inference time required to generate the k-th event be $\tau_{\text{infer}}(k)$, and let the actual acoustic duration of the generated event y_k be $D(y_k)$. To prevent the playback thread from exhausting the buffer while continuously reading generated events, the system expects that before the playback of the j-th event is completed, sufficient inference and scheduling margin has already been accumulated for the following event. This condition can be formalized as

$$\Delta T_{\text{buffer}} + \sum_{k=1}^{j} D(y_k) > \sum_{k=1}^{j+1} \tau_{\text{infer}}(k), \quad \forall j \in \{1, 2, \ldots, n-1\}. \tag{34}$$

Equation (34) states that the initial buffer plus accumulated acoustic duration should exceed the inference time needed for upcoming events. The mechanism does not eliminate model latency; it reduces perceived waiting by overlapping decoding and playback and by outputting the first MIDI events earlier than a fully blocking pipeline.

4.3.1 Speculative Preload for Candidate Endpoints

The revocable endpoint mechanism reduces false phrase endings, but its confirmation window adds latency. If Transformer inference starts only after $\Delta T_{\text{confirm}}$ expires, the confirmation time becomes part of the perceived first-note delay.

Speculative preload turns the *Candidate* state into a warm-up interval. When a candidate endpoint is marked, the system starts background inference so that initial tokens may already be available if the endpoint is committed.

When the system first detects that

$$t_{\text{current}} - T_{\text{last}} > \tau_{\text{cutoff}}, \tag{35}$$

the system records $T_{\text{candidate}}$, snapshots the current Call, and starts a speculative preload thread for forward propagation, context loading, and initial token generation. During the confirmation window, MIDI monitoring continues and branch selection occurs at $T_{\text{candidate}} + \Delta T_{\text{confirm}}$.

(1) Commit Branch. If no Note-On arrives, the endpoint is committed and the scheduler can reuse any initial tokens already produced by preload. The perceived first-note response latency can be approximated as

$$\tau_{\text{perceived}} = \max\left(0,\, \tau_{\text{first}} - \Delta T_{\text{confirm}}\right). \tag{36}$$

Here, τ_{first} denotes the time required by the model to generate the first playable token under normal inference conditions. If

$$\tau_{\text{first}} \leq \Delta T_{\text{confirm}}, \tag{37}$$

then first-token generation is largely covered by the confirmation window, allowing faster MIDI playback after commit.

(2) Cancel Branch. If a Note-On arrives, the candidate is canceled, temporary preload results are discarded, the new event is appended to the Call, and the system returns to *Listening*. Speculative preload is therefore a computation-latency trade-off: committed candidates benefit from early inference, while canceled candidates waste some temporary work. Together with the revocable endpoint mechanism, it balances endpoint safety against response immediacy.

4.4 Relative Interval Representation and Nucleus Sampling Control in a High-Entropy Setting

Without a fixed tonal constraint, the Transformer generates within the full twelve-tone pitch space. This preserves improvisational freedom but increases output entropy, so the system uses nucleus, or Top-p, sampling to reduce low-probability abnormal candidates [4].

(1) Relative event representation at the input side.

The input pipeline converts absolute MIDI timestamps into relative arrival times:

$$\Delta t_i = t_i - t_{i-1}. \tag{38}$$

This representation reduces dependence on fixed bars and global beat positions, emphasizing local rhythmic density, inter-event intervals, and micro-timing. Pitch organization is likewise represented through relative interval relationships, for example

$$\Delta p_i = p_i - p_{i-1}. \tag{39}$$

Together, relative timing and interval features make the prompt less tied to absolute positions or fixed tonal labels and more focused on motives, contours, and local tension in the Call.

(2) Nucleus sampling and temperature control at the output side.

During decoding, direct sampling from the full distribution can produce long-tail candidates such as abrupt leaps or local breaks. The system therefore applies nucleus sampling and temperature scaling as an external probability-control layer. Let $P(x)$ denote the model distribution

over candidate notes or event tokens. The system sorts candidates by probability and selects the smallest set S_p satisfying

$$S_p = \{x_{(1)}, x_{(2)}, \ldots, x_{(K)}\}, \tag{40}$$

where

$$K = \min \left\{ k \;\middle|\; \sum_{i=1}^{k} P(x_{(i)}) \geq p \right\}. \tag{41}$$

The threshold p is typically around 0.95. The system keeps the high-probability nucleus whose cumulative mass reaches this threshold, renormalizes within S_p, and samples from it:

$$P'(x) = \begin{cases} \dfrac{P(x)}{\sum_{y \in S_p} P(y)}, & x \in S_p, \\ 0, & x \notin S_p. \end{cases} \tag{42}$$

Meanwhile, the system uses the temperature parameter T to adjust the smoothness of the probability distribution. If the model outputs logits z_i, the temperature-scaled probability can be written as

$$P_T(x_i) = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}. \tag{43}$$

When $T < 1$, the distribution sharpens and favors stable high-probability candidates; when $T > 1$, it flattens and becomes more exploratory. Nucleus sampling and temperature scaling do not impose harmonic rules, but they reduce long-tail sampling while preserving open chromatic generation.

4.5 Phrase-level Music Control Layer

The Transformer operates through token-level continuation, while musical phrases also require duration balance, repetition control, and fallback behavior when no valid notes are produced. The Phrase-level Control Layer bridges this gap by analyzing the Call, constructing a response plan, and applying external constraints before and after generation. It does not modify model parameters or the forward probability distribution.

4.5.1 CallProfile-based Phrase Intention Analysis and ResponsePlan Construction

Phrase-level control first extracts a macroscopic representation from the Call. After endpoint detection closes the input window, the system maps

$$C = \{e_1^c, e_2^c, \ldots, e_m^c\} \tag{44}$$

into the CallProfile feature vector:

$$P_{\text{call}} = \left[T_{\text{call}}, N_{\text{call}}, \Omega_{\text{pitch}}, \bar{p}_{\text{call}}, H_{\text{pitch}}, \nabla_{\text{melody}}, \rho_{\text{rhythm}}, T_{\text{repeat}}\right]^T. \tag{45}$$

The components are:

- T_{call} denotes the duration of the Call phrase.
- N_{call} denotes the total number of notes.
- $\Omega_{\text{pitch}} = [p_{\min}, p_{\max}]$ represents the pitch range.
- $\bar{p}_{\text{call}}$ is the mean absolute pitch.
- H_{pitch} denotes the pitch-class histogram and the proportion of the dominant pitch.
- ∇_{melody} describes the first-order melodic contour.

- $\rho_{\text{rhythm}} = N_{\text{call}}/T_{\text{call}}$ measures the rhythmic density.
- T_{repeat} marks whether the ending of the Call contains a repeated-note gesture.

The control layer then constructs a ResponsePlan:

$$\text{Plan}_{\text{resp}} = \{T_{\text{target}}, N_{\text{target}}, \Omega_{\text{allow}}, CPR_{\text{limit}}, Share_{\max}, K_{\text{resample}}\}. \tag{46}$$

The ResponsePlan defines global boundaries outside the neural model. It estimates target duration and note count from the Call, anchors the allowed pitch range near the Call range and consonant intervals, and sets repetition thresholds through CPR_{limit} and $Share_{\max}$.

4.5.2 Repetition-tail Compression and Repetition Suppression

Repeated pitches or chordal gestures often signal phrase endings, but autoregressive models can over-attend to this recent tail and reproduce it excessively. The control layer therefore intervenes on both the input and output sides.

(1) Input-side prompt cleaning. Before inference, prompt cleaning marks a repeated final segment as an ending gesture when the same pitch exceeds a threshold such as $N_{\text{repeat}} \geq 3$ or dominates the final time window. The prompt then keeps at most two ending notes, preserving closure while reducing repetition bias.

(2) Output-side repetition constraint and resampling. During autoregressive generation, the system combines nucleus sampling with state-based rejection checks. Let the partial response before step j be $R_{<j}$. For candidate event e_j^r, two constraints are checked.

The first constraint is the Continuous Pitch Repetition constraint. If the candidate event causes the same pitch p to appear continuously more than CPR_{limit} times, the token is rejected.

The second constraint is the Dominant Share constraint. The system dynamically calculates the pitch distribution of $R_{<j}$. If accepting the current candidate would cause a single pitch to exceed the maximum allowed proportion $Share_{\max}$, the token is also rejected.

Invalid tokens are resampled up to K_{resample} attempts. If valid sampling still fails, the pitch is shifted by semitone steps toward a nearby consonant pitch around $\bar{p}_{\text{call}}$, helping the response escape a local repetition attractor while remaining related to the Call.

4.5.3 Motif-transformation Fallback and Duration Matching

(1) Fallback on empty generation. With sparse or unstable prompts, the model may return no playable notes. To avoid silence in the interaction loop, the system switches to a deterministic Motif Transformation Fallback Generator that transforms the ending motif of the Call:

$$R_{\text{fallback}} = T_{\text{scale}} \circ T_{\text{clip}} \circ T_{\text{shift}} \circ T_{\text{inverse}}(C_{\text{tail}}). \tag{47}$$

The process includes inversion, transposition toward $\bar{p}_{\text{call}}$, pitch-boundary clipping, and proportional rhythmic scaling. It is an engineering safety net rather than the primary source of musical creativity.

(2) Response duration matching. Generated note count does not guarantee an appropriate phrase length, because note durations and inter-onset intervals are stochastic. Before rendering, duration matching computes the raw response duration:

$$T_{\text{raw}} = t_n^r - t_1^r. \tag{48}$$

It then compares this value with the planned target duration T_{target} and defines the duration matching ratio as

$$\gamma = \frac{T_{\text{target}}}{T_{\text{raw}}}. \tag{49}$$

If γ falls outside a safe range, for example

$$\gamma \notin [0.85, 1.25], \tag{50}$$

the control layer stretches or compresses timing:

$$t^r_{i,\text{actual}} = t^r_1 + \alpha_{\text{stretch}} \cdot \left(t^r_{i,\text{raw}} - t^r_1\right), \tag{51}$$

and

$$d^r_{i,\text{actual}} = \alpha_{\text{stretch}} \cdot d^r_{i,\text{raw}}. \tag{52}$$

Here, α_{stretch} is bounded to prevent excessive timing distortion. The logging module records raw and adjusted response duration, the duration-match ratio, and the stretch factor, making duration matching both a control mechanism and an evaluable module in Section 5.

4.6 Chromatic Free Improvisation and Optional Style Constraint Layer

The default design goal is Chromatic Free Improvisation: the Transformer generates within the full twelve-tone pitch space, conditioned on the Call's pitch relationships, timing, and local motifs rather than a fixed scale. Some performance or evaluation scenarios, however, require stronger control over scale, meter, phrase length, strong beats, and cadence. The Optional Style Constraint Layer provides this control as post-processing through symbolic projection, temporal quantization, and structural correction. It is configurable and does not modify Transformer parameters or its forward probability distribution.

4.6.1 Chromatic Mode

Chromatic Mode is the default open-generation mode. It imposes no hard scale, mode, or rhythmic-grid constraints, allowing chromatic motion, tonal deviation, irregular rhythm, and free phrase development beyond bar-line boundaries. Control is still present through nucleus sampling, temperature scaling, endpoint detection, streaming decoding, and phrase-level constraints; what is removed is only the hard tonal or metrical projection.

4.6.2 Engineering Implementation of the pentatonic_2bar_4_4 Style Constraint Layer

For phrase-response forms, pentatonic-style generation, and reproducible evaluation, the system implements an optional `pentatonic_2bar_4_4` mode. It is a configurable post-processing style option rather than a replacement for Chromatic Mode.

The style constraint layer consists of the following five cascaded mechanisms:

1. **4/4 Meter Clock and Two-Bar Temporal Boundary**
   The mode introduces a 4/4 metrical clock and restricts the target response to two bars. If BPM is known, this equals eight quarter-note beats. Events beyond the boundary are truncated so that responses keep a stable experimental length.

2. **Pentatonic Scale Snapping and Pitch-Class Projection**
   The mode determines a pentatonic pitch set from the dominant Call pitch or a predefined tonal center:

   $$P_{\text{pentatonic}} = \{p \in \mathbb{Z} \mid (p \bmod 12) \in M_{\text{mode}}\}, \tag{53}$$

   where M_{mode} denotes the pitch-class set of the current pentatonic mode. When a generated pitch p_i does not belong to this set, the post-processing layer projects it to the nearest or musically more appropriate pentatonic scale degree:

   $$p'_i = \Pi_{P_{\text{pentatonic}}}(p_i). \tag{54}$$

This projection constrains output pitches at the post-processing level and does not imply that the model itself guarantees pentatonic modal logic.

3. **Rhythmic Grid Quantization**
   The two-bar temporal space is divided into a rhythmic grid, such as sixteenth notes or eighth-note triplets. Each event time and duration is mapped to the nearest grid point:

$$t^r_{i,\text{quantized}} = \Delta t_{\text{grid}} \cdot \text{round}\left(\frac{t^r_{i,\text{raw}}}{\Delta t_{\text{grid}}}\right). \tag{55}$$

   Durations can be quantized in the same way. This reduces micro-timing freedom but improves rhythmic regularity and comparability.

4. **Strong-Beat Stability Control**
   Strong beats, such as beats one and three, are checked for modal stability. Unstable strong-beat notes may be moved toward the tonic, dominant, or another stable scale degree, while weaker-beat passing tones are more likely to remain.

5. **Cadential Pitch-Class Resolution and Maximum Melodic Leap Restriction**
   Final pitch classes are encouraged to resolve to stable degrees such as the tonic or dominant, reducing unresolved endings caused by truncation. The system also limits adjacent melodic leaps. Let the maximum allowed leap be $\Delta p_{\max}$. When

$$|p^r_i - p^r_{i-1}| > \Delta p_{\max}, \tag{56}$$

   the layer applies interval clipping, octave folding, or nearby scale-degree replacement to keep the contour playable and continuous.

In summary, the optional style layer provides a reproducible style-focused mode through temporal boundaries, pentatonic projection, rhythmic quantization, strong-beat control, and cadence correction. It extends controllability for specific tasks without replacing Chromatic Free Improvisation or changing the underlying Transformer.

5 Experimental Design and Objective Evaluation

This section evaluates the proposed adaptation method through a three-layer evidence chain: candidate strategy comparison, module attribution, and runtime scheduling. The goal is not to replace human listening judgment with a single objective score, but to establish reproducible structural and scheduler-level evidence for the system.

5.1 Experiment Hierarchy and Evaluation Goals

The evaluation is organized to avoid a single-score interpretation of the system. Because the proposed method combines musical control, module-level engineering, and runtime scheduling, each claim is tested at the level where it can be observed most directly. Figure 2 summarizes this structure before the individual datasets, metrics, and results are introduced.

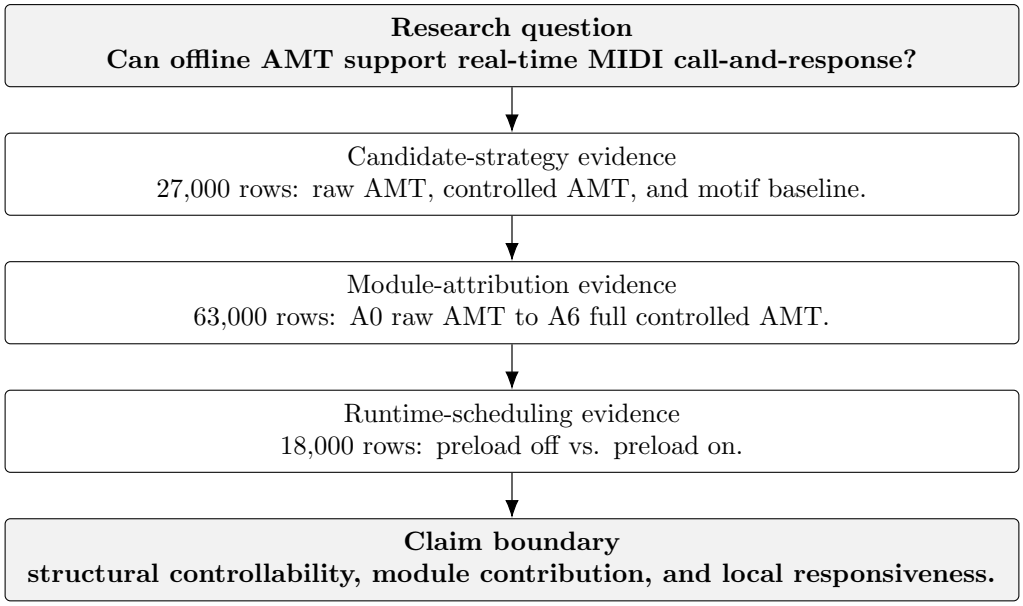

Figure 2: Experiment hierarchy and evidence chain.

The first level asks whether controlled AMT improves over raw AMT and how it compares with a rule-based motif baseline. The second level attributes the controlled-AMT gain to individual modules, and the third level tests whether speculative preload improves endpoint-to-first-MIDI latency under the logged local scheduler. These layers extend, rather than replace, the original three-strategy Call100 comparison. The value of `amt_small_controlled` should therefore be described as improved controllability relative to raw AMT, not as dominance over every baseline or metric.

5.2 Construction of the Call100 Benchmark

Call100 contains 100 call phrases and extends early demonstrations into a mixed-distribution robustness test. It is built in two stages. First, it reuses Call50 without modifying its files: 10 artificial stress-test calls and 40 real public MIDI fragments. The artificial calls deliberately probe likely failure modes, including clear motifs, minor-pentatonic questions, blues or chromatic tension, half cadences, repeated tails, syncopation, sparse long tones, dense eighth-note motion, arch contours, and large leaps followed by stepwise filling.

The 40 real calls in Call50 are extracted from POP909 [14] using a conservative two-bar melody-sampling procedure. The extraction script scans MIDI tracks with melody-like names such as `melody`, `main`, `vocal`, and `lead`, while excluding tracks that are more likely to represent accompaniment or percussion, such as `chord`, `bass`, and `drum`. Candidate windows are two bars long, corresponding to 8 beats, and the window advances by 2 beats. A candidate is kept only if it contains between 6 and 24 notes, has a polyphonic-overlap ratio no greater than 0.10, and reaches a major/minor scale-fit score of at least 0.85. A fixed random seed, 20260529, is then used to shuffle the candidates and select 40 real snippets reproducibly.

Second, Call100 copies Call50 into a new benchmark directory and adds 50 public MIDI excerpts without rewriting the original `calls_50` directory. In the reproducible run reported here, the available public source was POP909, so all additions come from POP909. The builder constructs two-bar windows from tracks such as `MELODY`, `BRIDGE`, and `PIANO`, computes descriptive features, scores candidates against eight diagnostic target categories, and applies fingerprint deduplication.

Table 1 summarizes the source distribution, Call50 origins, and public-addition targets. The benchmark is diagnostic rather than corpus-representative: it preserves artificial stress

cases while adding public excerpts for broader melodic input. Planned MAESTRO, ASAP, Lakh/Slakh, and GiantMIDI sources were not locally available, so the dataset report records their absence and the public additions are backfilled from POP909.

Table 1: Detailed composition of the Call100 benchmark

Source or origin	Subset or target type	Calls	Purpose or selection rule
`calls_50`	Artificial stress tests	10	Hand-designed edge cases: pentatonic motif, minor question, blues/chromatic tension, half cadence, repetition tail, syncopation, sparse long tones, dense motion, arch contour, and leap gap filling.
`calls_50`	Real POP909 snippets	40	Two-bar monophonic melody windows from melody-like tracks; 6–24 notes, overlap ratio ≤ 0.10, scale fit ≥ 0.85, seed 20260529.
`POP909`	Clear motif	12	Public additions selected for stable motivic structure.
`POP909`	Expressive rubato	8	Irregular timing and flexible inter-onset intervals.
`POP909`	Chord/arpeggio texture	8	Chordal or partially polyphonic input gestures.
`POP909`	Sparse short phrase	5	Low-information calls with few notes or long gaps.
`POP909`	Dense fast motion	5	High onset density and faster melodic movement.
`POP909`	Chromatic outside notes	5	Lower scale-fit or chromatic-tension cases.
`POP909`	False-ending pause	4	Internal or trailing pauses that may confuse endpoint detection.
`POP909`	Repetition tail	3	Repeated endings that may trigger response repetition.
Total	Call100 benchmark	100	10 artificial stress calls, 40 real Call50 snippets, and 50 public POP909 additions.

5.3 Candidate Strategies and Parameter Presets

The formal experiment compares three candidate strategies, summarized in Table 2. The raw AMT condition directly uses the AMT generator without external phrase-level control. The controlled AMT condition keeps the same generator but adds call profiling, prompt cleaning, repetition suppression, fallback generation, and duration matching. The motif baseline does not use a neural generator; instead, it produces responses through rule-based motif transformation and is treated as a strong structural baseline.

The experiment uses six parameter presets, listed in Table 3. Each call, preset, and candidate combination is run for 15 trials. The total experiment scale is therefore:

$$100 \text{ calls} \times 6 \text{ presets} \times 3 \text{ candidates} \times 15 \text{ trials} = 27000. \tag{57}$$

Each preset/candidate pair contains 1500 samples. The validation file confirms that the full objective-results table contains exactly 27,000 rows, the number of preset/candidate/call groups is 1800, the validation summary has status `passed`, and there are no empty objective scores, missing response MIDI files, or unknown call IDs.

Table 2: Candidate strategies in the Call100 experiment

Candidate ID	Description
`amt_small_raw`	Raw AMT response generation without external phrase-level control.
`amt_small_controlled`	AMT response generation with call profiling, repetition suppression, fallback generation, and duration matching.
`motif_transform_baseline`	Rule-based motif transformation baseline used as a strong non-neural comparison.

Table 3: Parameter presets used in the Call100 experiment

Preset group	Preset ID
No-theory control	`pentatonic_no_theory`
Balanced setting	`pentatonic_balanced`
Conservative setting	`pentatonic_conservative`
Creative setting	`pentatonic_creative`
Wide creative setting	`pentatonic_creative_wide`
Low-temperature setting	`pentatonic_low_temp_no_strongbeat`

5.4 Objective Metric System

To avoid evaluating generated music with only a single similarity score or a small number of isolated indicators, the objective evaluation is organized into six groups of structural proxy metrics, as shown in Table 4. These metrics cover tonal stability, rhythmic quality, melodic motion, repetition control, pitch diversity, and structural redundancy.

Table 4: Objective metric groups

Metric group	Representative metrics
Tonality	pitch-scale rate (PSR), strong-beat stable rate, cadence score
Rhythm	qualified note rate (QN), qualified rhythm rate (QRF), groove similarity
Melodic motion	stepwise rate, tone span ratio, maximum interval
Repetition control	consecutive pitch repetition (CPR), longest repeat run
Pitch diversity	pitch-class histogram entropy (PCHE), used pitch classes (UPC)
Structural redundancy	compression ratio

The overall `objective_score` is computed as a weighted combination of these six categories:

$$\begin{aligned}\text{objective\_score} = \text{weighted\_sum}(&\text{tonality}, \text{rhythm}, \text{interval},\\ &\text{repetition}, \text{pitch\_diversity}, \text{compression}).\end{aligned} \tag{58}$$

The `objective_score` used in this work emphasizes structural stability, controllability, and generation robustness. It is therefore appropriate for candidate screening and failure-mode analysis, but it should not be treated as a standalone substitute for human listening evaluation. In other words, the objective metrics are reproducible proxies for structural quality rather than complete replacements for perceived musicality [6, 8].

5.5 Call100 Leaderboard and Result Analysis

Table 5 reports the top five preset/candidate pairs in the formal Call100 experiment. The two highest-scoring entries are both motif-baseline settings, while the next three entries are controlled-AMT settings. The best overall mean objective score is 0.766679, and the best controlled-AMT score is 0.764335. This ranking already suggests the central result of the experiment: external control makes AMT much stronger than its raw version, but the rule-based motif baseline remains highly competitive under the current structural proxy score.

Table 5: Top five entries in the Call100 objective leaderboard

Rank	Preset label	Candidate strategy	Mean score	Samples
1	Balanced	Motif baseline	0.766679	1500
2	Conservative	Motif baseline	0.766142	1500
3	Conservative	Controlled AMT	0.764335	1500
4	Low-temp, no strong beat	Controlled AMT	0.762782	1500
5	Balanced	Controlled AMT	0.760900	1500

Panel (a) of Figure 3 confirms the same pattern as the table: motif-baseline settings occupy the top two ranks, while controlled AMT occupies ranks 3–5 and clearly outperforms the best raw AMT setting. Unlike the earlier Call50 leaderboard, where a low-temperature controlled-AMT setting led, the larger Call100 distribution shows that the rule-based motif transformation baseline remains a strong comparison under the current structural proxy metrics.

Table 6 compares several key metrics for rank 1, rank 3, and the best raw AMT combination. The motif baseline achieves a higher tonality score, reflecting the advantage of rule-based pitch organization and cadence control. Controlled AMT performs strongly on duration matching, interval score, and compression score, reflecting the effect of external response-length and structural-shape control. Raw AMT is competitive on some interval-related metrics, but its overall repetition, compression, and note-count behavior are less stable.

Table 6: Detailed metric comparison among representative combinations

Combination	Obj.	Tonal.	Rhythm	Interval	Repet.	Dur. match
Rank-1 motif baseline	0.766679	0.972630	0.768430	0.682771	0.753321	0.890467
Rank-3 controlled AMT	0.764335	0.903633	0.763440	0.741680	0.726267	0.938038
Best raw AMT	0.696652	0.911404	0.704589	0.761084	0.542461	0.897567

5.6 Overall Candidate Comparison and Statistical Significance

Table 7 summarizes the overall performance of the three candidate strategies over 9000 samples each. The motif baseline obtains the highest overall mean objective score, 0.752682. Controlled AMT obtains a mean score of 0.747048, which is substantially higher than the raw AMT mean score of 0.683141.

Table 7: Overall comparison of candidate strategies

Candidate strategy	Samples	Mean score	Interpretation
Motif baseline	9000	0.752682	Structurally stable, strong in tonality and cadence, and a strong baseline under the current objective metrics.
Controlled AMT	9000	0.747048	Clearly better than raw AMT, with better duration matching, rhythmic validity, and compression structure, but with frequent fallback activation.
Raw AMT	9000	0.683141	Without external control, repetition, extreme movement, and structural redundancy are more visible.

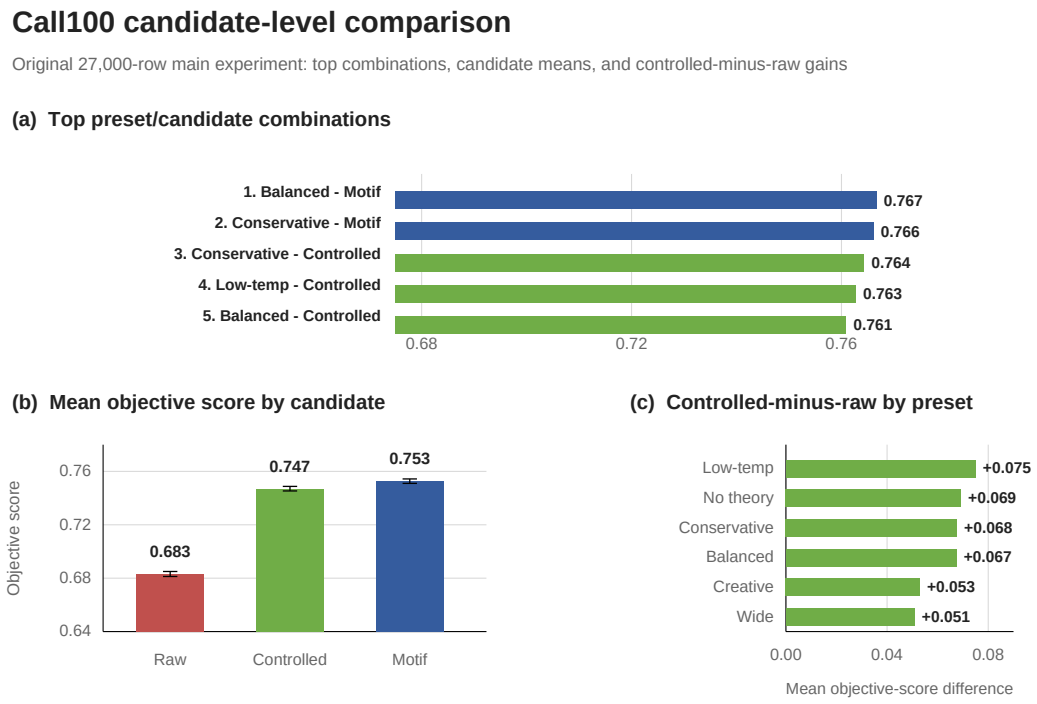

Figure 3: Call100 candidate-level comparison. Panel (a) shows the top preset/candidate combinations, panel (b) shows mean objective score with 95% confidence intervals by candidate strategy, and panel (c) shows controlled-minus-raw improvement by preset.

Panel (b) of Figure 3 separates raw AMT from the controlled and rule-based strategies. The paired bootstrap tests in Table 8 quantify this result: controlled AMT improves over raw AMT by +0.063907 on average, with a 95% confidence interval of [0.061668, 0.066294] and bootstrap $p < 10^{-6}$. Controlled AMT remains slightly below the motif baseline overall, with a mean difference of -0.005633 and a 95% confidence interval of [-0.007882, -0.003501].

Table 8: Paired bootstrap tests between candidate strategies

Comparison	Mean difference	95% CI	bootstrap p
`controlled - raw`	+0.063907	[0.061668, 0.066294]	$< 10^{-6}$
`controlled - motif`	-0.005633	[-0.007882, -0.003501]	$< 10^{-6}$
`motif - raw`	+0.069541	[0.067351, 0.072127]	$< 10^{-6}$

Panel (c) shows that controlled AMT improves over raw AMT under every preset, with the largest gain in the low-temperature setting. The conservative conclusion is therefore that external phrase-level control reliably improves raw AMT outputs, while the motif baseline remains highly competitive under objective proxies that emphasize tonal stability, rhythmic regularity, and repetition control.

5.7 Controlled-AMT Module Ablation

The candidate-level Call100 experiment shows that controlled AMT improves over raw AMT, but not which components drive the gain. A module-level ablation therefore uses the same 100 calls, six presets, seven variants, and 15 trials for each preset/variant/call combination:

$$100 \times 6 \times 7 \times 15 = 63000. \tag{59}$$

The final validation confirms 63,000 actual rows, 4200 preset/variant/call groups, 15 trials per group, no empty objective scores, no missing response MIDI files, no unknown call IDs, and validation status `passed`.

Table 9: Controlled-AMT ablation variants

Variant	Module added	Description
A0	Raw AMT	No external control.
A1	Prompt cleaning	Only tail-repeat compression and prompt cleaning are enabled.
A2	Repetition suppression	Adds generation-time repetition suppression and resampling.
A3	Duration matching	Adds response duration and note-count matching.
A4	Motif fallback	Adds motif fallback for empty or invalid neural output.
A5	Style constraint	Adds pentatonic, two-bar, and 4/4 response style constraints.
A6	Full controlled	All controlled-AMT modules are enabled.

Table 10 reports mean structural scores over 9000 samples per variant. Full controlled AMT improves the mean objective score from 0.560968 to 0.732610, an absolute gain of 0.171642. The largest single change is associated with the style constraint layer; repetition suppression and duration matching also contribute before final integration.

Table 10: Ablation summary by variant over Call100

Variant	Obj. score	CPR	Dur. match	PSR	Cadence
A0 raw AMT	0.560968	0.152200	0.823200	0.568857	0.427850
A1 prompt cleaning	0.574265	0.131736	0.822865	0.572559	0.440333
A2 repetition suppression	0.610704	0.036216	0.835172	0.538713	0.384656
A3 duration matching	0.640526	0.049203	0.826190	0.538235	0.384544
A4 fallback	0.640526	0.049203	0.826190	0.538235	0.384544
A5 style constraint	0.708929	0.128617	0.844393	1.000000	0.731233
A6 full controlled	0.732610	0.135255	0.936885	1.000000	0.995389

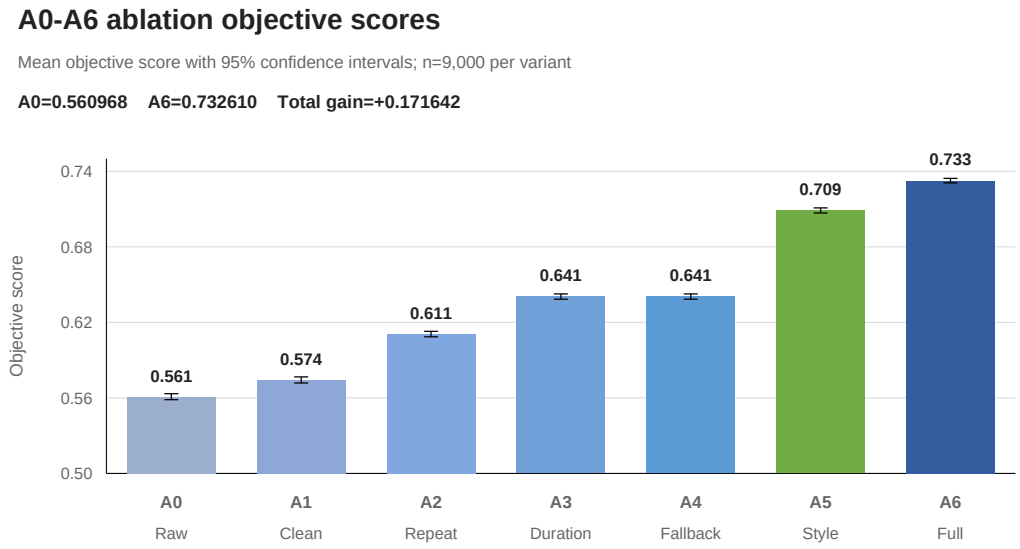

Figure 4: A0–A6 module-ablation objective scores. Bars report mean objective score and 95% confidence intervals over 9000 samples per variant.

Figure 4 shows the same trajectory: A0–A4 improve gradually and plateau at motif fallback, while A5 and A6 produce the dominant final increases.

Table 11: Stepwise module contributions to objective score

Step	Module added	Mean delta	95% CI	p
A1–A0	Prompt cleaning	0.013297	[0.010832, 0.015736]	$< 10^{-6}$
A2–A1	Repetition suppression	0.036439	[0.034068, 0.038919]	$< 10^{-6}$
A3–A2	Duration matching	0.029822	[0.028143, 0.031398]	$< 10^{-6}$
A4–A3	Motif fallback	0.000000	[0.000000, 0.000000]	1.000000
A5–A4	Style constraint	0.068403	[0.066357, 0.070537]	$< 10^{-6}$
A6–A5	Full controlled	0.023681	[0.022480, 0.024868]	$< 10^{-6}$

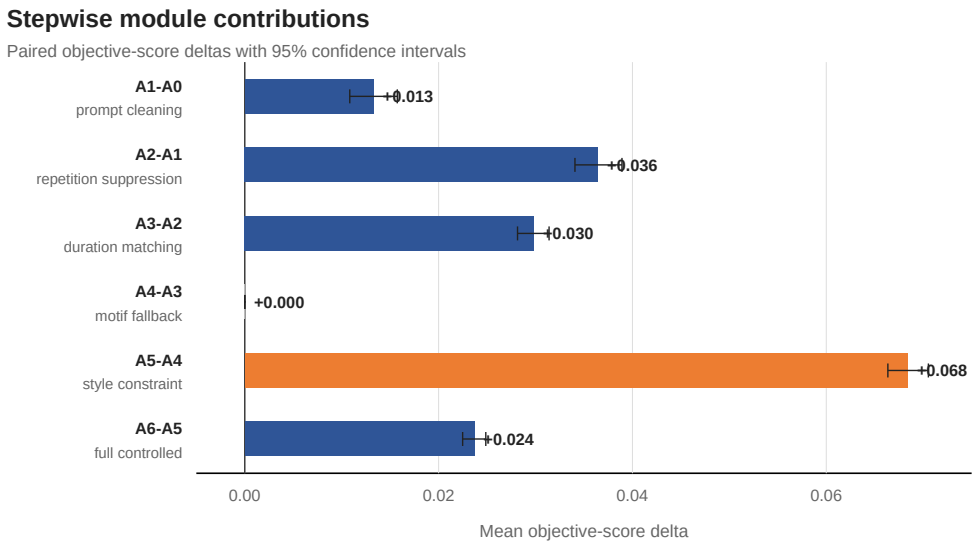

Figure 5: Stepwise module contribution plot. Horizontal bars show paired mean objective-score deltas and 95% confidence intervals for each module addition.

Table 11 and Figure 5 give paired stepwise deltas. The style-constraint gain reflects symbolic projection toward the pentatonic two-bar setting, not intrinsic Transformer mastery of that style. The zero A4–A3 delta means that, in this run, motif fallback acted mainly as a safety mechanism for empty or invalid outputs rather than a driver of average objective-score improvement.

5.8 Runtime Latency Logging and Speculative Preload

The objective evaluation above measures structural quality, but it does not directly test the real-time response behavior of the system. Therefore, a runtime latency logging experiment was conducted on the same local development laptop used for system testing: a Lenovo 82JQ running 64-bit Windows 11 Pro, with an AMD Ryzen 7 5800H CPU, 16 GB RAM, and an NVIDIA GeForce RTX 3060 Laptop GPU with 6 GB VRAM. The logged inference timings come from actual local AMT generation during the A6 full-controlled ablation runs, and MIDI output timing is represented by the local playback-scheduler model. The two conditions are defined as follows: L0 disables speculative preload and starts inference after endpoint commit; L1 enables speculative preload during the candidate endpoint window. The preload window is 250 ms, the micro-buffer is 80 ms, and the underrun deadline is 80 ms.

The primary latency metric is endpoint-to-first-MIDI latency:

$$L_{\text{first}} = t_{\text{first\_midi\_out}} - t_{\text{endpoint\_commit}}. \tag{60}$$

This metric measures the delay between the confirmed phrase endpoint and the first scheduled MIDI response event.

Table 12: Runtime latency comparison between preload-off and preload-on conditions

Condition	Samples	Mean ms	P50 ms	P95 ms	P99 ms	Underrun
L0 preload off	9000	161.303	136.455	323.521	479.574	20.744%
L1 preload on	9000	85.534	80.000	80.000	229.574	3.556%

Table 13: Paired latency reduction from speculative preload

Comparison	Paired samples	Mean reduction ms	95% CI
L1 preload on vs. L0 preload off	9000	75.768	[74.724, 76.793]

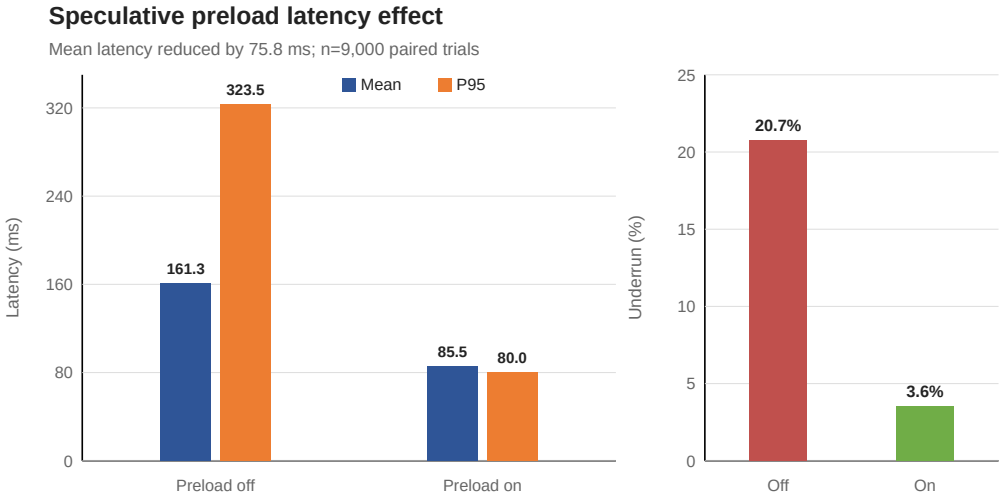

Figure 6: Preload off/on latency comparison. The left panel reports mean and P95 endpoint-to-first-MIDI latency; the right panel reports buffer-underrun rate.

The paired test gives $p < 10^{-6}$, indicating a consistent latency reduction in the logged local deployment setting. Preload reduces mean latency from 161.303 ms to 85.534 ms, collapses P95 latency to the 80 ms micro-buffer boundary, and lowers buffer-underrun rate from 20.744% to 3.556%. This improves scheduled post-endpoint responsiveness by moving early inference into the candidate-confirmation interval; it does not reduce intrinsic AMT generation time.

5.9 Dataset Source Robustness and Failure-Case Analysis

The grouped statistics of Call100 make it possible to examine whether the results are driven only by a small number of easy samples. Table 14 shows that the original Call50 subset obtains a mean score of 0.740086, higher than the added POP909 subset score of 0.715161. By origin, `artificial_stress` and `real_public_dataset` obtain similar scores, 0.742780 and 0.739413 respectively. The `public_midi_extract` group scores lower, at 0.715161.

Table 14: Performance by source dataset and origin

Grouping field	Value	Samples	Mean score
source dataset	`calls_50`	13500	0.740086
source dataset	`POP909`	13500	0.715161
origin	`artificial_stress`	2700	0.742780
origin	`real_public_dataset`	10800	0.739413
origin	`public_midi_extract`	13500	0.715161

This difference does not indicate failure on POP909; it shows that the added public excerpts are harder. The POP909 subset has a higher average maximum interval, 12.093 versus 8.360

for calls_50, and a higher average note count, 15.905 versus 13.588, increasing the difficulty of response planning and duration matching.

The category-level results provide a similar picture. In Table 15, pentatonic_clear_motif and modal_question obtain the highest mean scores, suggesting that clear motifs and explicit modal-question structures are easier for the current control strategy to handle. Lower-scoring categories such as sparse_cadential, sparse_short, and expressive_rubato reveal weaknesses in rhythm estimation, duration matching, and structural filling for short, sparse, or rubato-like inputs.

Table 15: Examples of high- and low-scoring categories

Category	Samples	Mean score	Interpretation
pentatonic_clear_motif	270	0.787362	Clear motif and stable scale structure.
modal_question	270	0.787159	Clear question-like phrasing and cadential target.
syncopation	270	0.768083	Distinct rhythmic profile remains structurally manageable.
expressive_rubato	2160	0.691445	Free timing increases rhythm and duration-matching difficulty.
sparse_short	1350	0.667246	Low input information makes structural filling less stable.
sparse_cadential	270	0.665250	Sparse cadential fragments often lead to short or weak responses.

Failure-case diagnostics also require care. The experiment writes 5377 diagnostic rows, not 5377 generation failures. The most frequent label is fallback use, with 3555 rows; bottom-5% objective-score cases follow with 1225 rows, and poor duration matching accounts for 322 rows. For controlled AMT, the fallback rate is approximately 41.46%, and the mean repetition resampling count is approximately 1.639 per sample. These values measure control-layer intervention frequency, while the more concrete weak points are low objective scores, poor duration matching, and very low note-count outputs. Fallback frequency and A4–A3 ablation answer different questions: activation rate versus incremental mean-score effect.

5.10 Additional Result Patterns

The remaining summaries reinforce the main pattern: top entries are tightly clustered, confidence intervals separate raw AMT from controlled or rule-based strategies, controlled-minus-raw is positive across presets, and sparse, rubato-like, or short cadential inputs are harder than clear motif or modal-question inputs.

5.11 Interpretation and Scope of Evaluation

The scope of the evidence is limited. A0–A6 attributes the controlled-AMT gain mainly to style constraints, repetition suppression, and duration matching, while motif fallback functions primarily as a safety mechanism in this run. The preload experiment shows a scheduler-level latency reduction, not faster AMT inference. Because the paper does not include a formal blind listening test or live user study, perceptual preference and live interaction quality remain outside the verified claims.

6 Conclusion

This paper contributes an external adaptation method for using an offline AMT in real-time MIDI call-and-response without retraining. Its endpoint model, MIDI routing architecture, phrase-level controls, and speculative scheduler are evaluated as one deployment method rather than as separate claims.

The evidence is strongest at three levels: controlled AMT improves over raw AMT in the 27,000-row Call100 comparison; A0 to A6 improves mean objective score from 0.560968 to 0.732610 in the 63,000-row ablation; and preload reduces mean endpoint-to-first-MIDI latency from 161.303 ms to 85.534 ms while lowering buffer-underrun rate from 20.744% to 3.556% in the 18,000-row latency experiment. These results support structural controllability and local runtime responsiveness, but not universal superiority over rule-based baselines, faster intrinsic AMT inference, or listener preference. Those questions require listening tests, live co-performance studies, and broader deployment conditions.

AI Tool Usage Statement

AI tools were used in this project as auxiliary research, writing, and programming support. This disclosure refers to general-purpose AI assistance used during project preparation; the autoregressive Transformer system evaluated in this paper is part of the research object itself and is described separately in the methodology and experiments.

The authors used OpenAI ChatGPT/Codex in three main places. First, during software development, AI assistance was used to inspect Python and MIDI-processing code, identify possible implementation errors, debug LaTeX or Python issues, and draft small validation or analysis procedures. All code execution, experiment running, and result validation were performed and checked by the authors. Second, during manuscript preparation, AI assistance was used to improve English wording, reorganize paragraphs, refine LaTeX formatting, and make tables and figure descriptions clearer. Third, during result reporting, AI assistance was used to help summarize verified CSV and log outputs and convert checked numerical results into concise academic prose.

AI tools were not used to fabricate experimental data, replace the Call100 objective evaluation, make unsupervised final research decisions, or generate references accepted without checking. All numerical claims, tables, figures, citations, and final conclusions were reviewed by the authors against the local experiment outputs, source code, and cited literature.

References

[1] Nicolas Boulanger-Lewandowski, Yoshua Bengio, and Pascal Vincent. Modeling temporal dependencies in high-dimensional sequences: Application to polyphonic music generation and transcription. In *Proceedings of the 29th International Conference on Machine Learning*, pages 1159–1166, 2012.

[2] Daryl J. Daley and David Vere-Jones. *An Introduction to the Theory of Point Processes: Volume I: Elementary Theory and Methods*. Springer, New York, 2 edition, 2003.

[3] Douglas Eck and Jürgen Schmidhuber. Finding a temporal structure in music with lossy lstm recurrent networks. *Neural Computation*, 14(11):2607–2625, 2002.

[4] Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. The curious case of neural text degeneration. In *International Conference on Learning Representations*, 2020.

[5] Cheng-Zhi Anna Huang, Ashish Vaswani, Jakob Uszkoreit, Noam Shazeer, Ian Simon, Curtis Hawthorne, Andrew M. Dai, Matthew D. Hoffman, Monica Dinculescu, and Douglas

Eck. Music transformer: Generating music with long-term structure. In *International Conference on Learning Representations*, 2019.

[6] Shulei Ji, Xinyu Yang, and Jing Luo. A survey on deep learning for symbolic music generation: Representations, algorithms, evaluations, and challenges. *ACM Computing Surveys*, 56(1):1–39, 2023.

[7] Tornike Karchkhadze and Shlomo Dubnov. Towards real-time human-ai musical co-performance: Accompaniment generation with latent diffusion models and max/msp. arXiv preprint arXiv:2604.07612, 2026.

[8] Alexander Lerch, Claire Arthur, Nick Bryan-Kinns, Corey Ford, Qianyi Sun, and Ashvala Vinay. Survey on the evaluation of generative models in music. *ACM Computing Surveys*, 2025.

[9] Mark Nelson and Belinda Thom. A survey of real-time midi performance. In *Proceedings of the International Conference on New Interfaces for Musical Expression*, pages 35–38, 2004. doi: 10.5281/zenodo.1176643.

[10] Sageev Oore, Ian Simon, Sander Dieleman, Douglas Eck, and Karen Simonyan. This time with feeling: Learning expressive musical performance. *Neural Computing and Applications*, 32:955–967, 2020.

[11] Nicola Privato, Omar Rampado, and Alberto Novello. Scramble live: Combining lstm and markov chains for real-time musical interaction. In *Proceedings of the 19th Sound and Music Computing Conference*, 2022. doi: 10.5281/zenodo.6800786.

[12] John Thickstun, David Hall, Chris Donahue, and Percy Liang. Anticipatory music transformer. *arXiv preprint arXiv:2306.08620*, 2023.

[13] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[14] Ziyu Wang, Ke Chen, Junyan Jiang, Yiyi Zhang, Maoran Xu, Shuqi Dai, Xianbin Gu, and Gus G. Xia. Pop909: A pop-song dataset for music arrangement generation. In *Proceedings of the 21st International Society for Music Information Retrieval Conference*, pages 38–45, 2020.